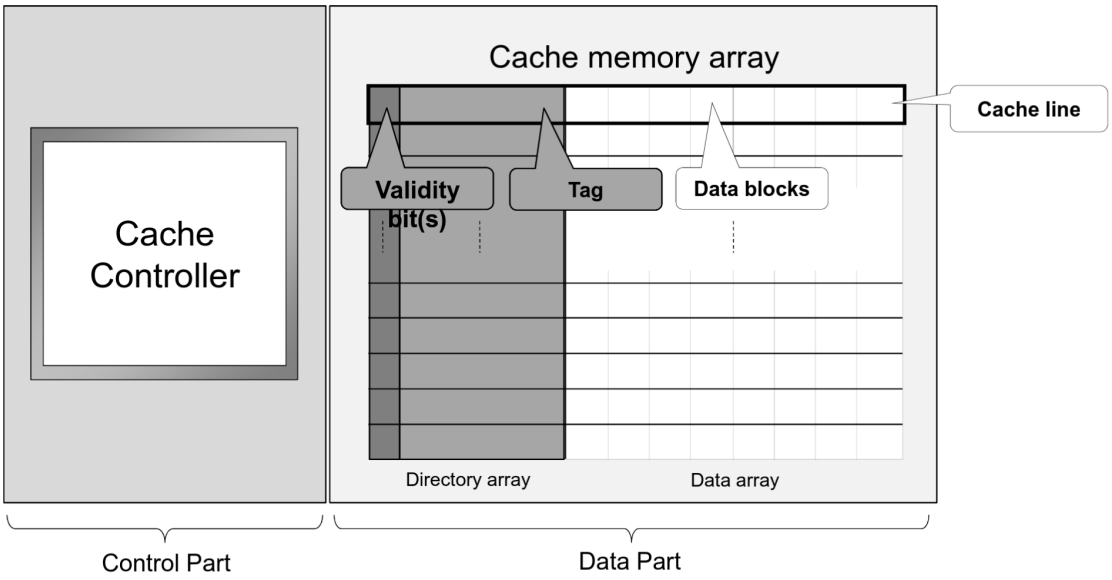
1 Lesson 27-10-26

Last lesson we introduced caches.

1.0.1 Direct mapped cache

We know look at how cache memories are actually realized in practice.
The first design design is the usual *direct mapped* cache:

- A cache is organized in **cache lines**;
- A line contains a memory block that includes some memory words;
- Each line is associated with a tag field, which indicates the memory block present in the line at that time.



We also need some **control logic** for:

- Intercepting the addresses produced by the processor;
- Checking inside the cache for the possible presence of the block that the processor wants to access;
- Possibly loading the block.

As we said, the mechanism of splitting the memory address in **tag**, **index** and **offset** is the usual direct mapping technique.

- The *advantage* of it is that the mechanism can be easily implemented in hardware (the least significant block identifier bits identify the cache line)
- The *disadvantage* is that if the program frequently accesses 2 blocks corresponding to the same cache line, a miss occurs at each memory access.

1.0.2 Fully associative mapping

This is the opposite approach of direct mapping: **fully associative mapping** (rarely implemented in practice). It is the case in which each block of the main memory can be stored in any cache line.

- The *advantage* is the maximum flexibility in choosing the cache line to use;
- The *disadvantag* is the complexity of search hardware (usually adopting associative memory such as **CAM**, *Content Addressable Memory*).

1.0.3 Set associative mapping

This is an implementation which expands on direct mapping, giving some of the benefits of associative mapping: **set associative mapping**.

The cache lines are subdivided into a number S of sets, consisting of W (ways) lines. We have that:

$$S = \frac{N}{W}$$

where N is the total number of cache lines: this means that our S sets map to our old memory blocks in the direct mapping cache, and each way is basically superimposed on the relative set.

A memory block i is associated with the set k, with:

$$k = i \,\%\, S$$

same as in the direct mapping sense, but the block i can be put into any of the W lines (ways) of the set k (some kind of associative memory should be used).

A set associative cache with W lines in each set is called a W-ways cache. Common values for W are the first powers of two.

Since, as we said, the memory block can be put into any of the ways, we need mechanisms to *retrieve* the correct block and *replace* the correct block on eviction:

- **Retrieval** is easy if the number of ways is limited: we can simply use W comparators per set, each checking on a specific way;
- **Eviction** can be done via a number of algorithms, mostly ideal with heuristic implementations. The eviction algorithm defines which cache line should be used to store a memory block, amongst those associated with the block (in the case of associative or set associative mapping). It is chosen from:
  - **LRU** (*Least Recently Used*): the most used;
  - **FIFO** (*First-In First-Out*): the cheapest;
  - **LFU** (*Least Frequently Used*): theoretically, the most effective;
  - Random: simple and effective.

1.0.4 Write policy

An important policy on the memory controller which must be chosen is **write policy**. 2 main options:

- **Write-back**: when a write is done on a memory location present in a cacheline, we only update the cache memory: the changes are *written back* to DRAM only at a later moment. Very quick and efficient, bad for parallel architectures (information goes out of sync);
- **Write-through**: when a write is done on a memory location present in cache, both the cacheline and DRAM are updated. Cache is always synchronized to DRAM, but this option is slower.